



CSE 598 008 EECS 498 003 Lab 5

Tianyiming (Henry) Jing
October 2, 2026

Based on slides by Ivan Wei (Fall 2025).

Reminder

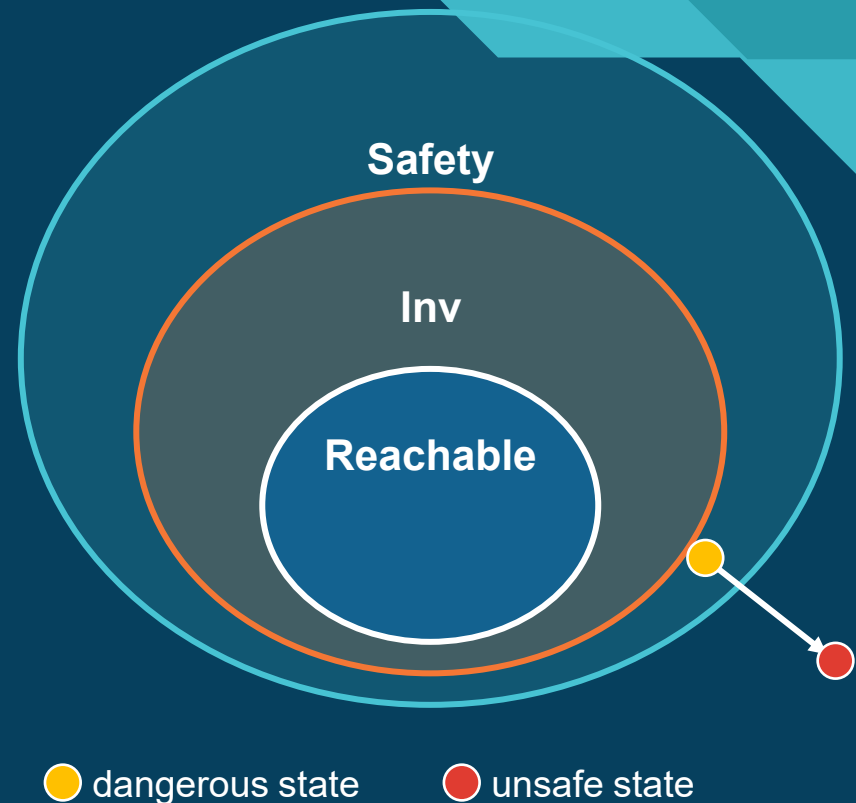
- Problem Set 2 (Chapters 3 and 4) due October 7, 11:59pm
- Problem Set 3 (Chapter 5) will be released on October 8
- Midterm will be on Thursday, October 22, 6pm – 8pm, FMCRB 1050
 - No lecture that day
 - Closed book; one double-sided cheat sheet allowed, 10pt minimum
 - Covers everything up to Chapter 4 (excluding distributed systems)

Question 1

- What do you think others in this class might be confused about or have questions about?
- Write your answer on the piece of paper

Dangerous States

- A dangerous state is safe, but one step away from an unsafe state
- It is not reachable, but Safety alone cannot rule it out
- This is why Safety is often not inductive
- An inductive invariant must:
 - leave out every dangerous state
 - still contain every reachable state



Modeling Traps

- Trivialization #1: a single variable

```
datatype Variables = Variables(whoHoldsTheLock: int)
```

- The state cannot even describe two clients holding the lock

- Trivialization #2: putting Safety or Inv in your transitions

```
predicate ReleaseLock(v) {  
  && Safety(v)  
  && ...  
}
```

- Bad steps are ruled out by decree, so the proof shows nothing

- A trivial model verifies, but says nothing about the real system

Proof Exercise

- Any questions about last week's problems? We can discuss them
- This week: Zero-Sum Society
- Starter code: `1ab05.dfy` on the course GitHub

Tips - Proof by Induction

```
ghost function CountPos(s: seq<int>): nat
{
  if |s| == 0 then 0
  else (if s[0] > 0 then 1 else 0) + CountPos(s[1..])
}
```

```
lemma CountPosAppend(s: seq<int>, t: seq<int>)
  ensures CountPos(s + t) == CountPos(s) + CountPos(t)
{
  if |s| == 0 {
    // Base case
    assert s + t == t;
  } else {
    // Inductive step: use the lemma on s[1..]
    assert (s + t)[1..] == s[1..] + t;
    CountPosAppend(s[1..], t);
  }
}
```

- The lemma copies the function's shape: the same if, the same smaller input $s[1..]$
- Calling the lemma on $s[1..]$ is how you use the induction hypothesis
- If it still fails, assert how the big input relates to the small one (a “magic” line)
- Unfold the function the way it folds: peel off $s[0]$, and the lemma on $s[1..]$ covers the rest

Question 2

- Which do you prefer?
 - The first three labs, where we head into the exercises directly
 - The recent two labs, where we go over some key concepts from the week's lecture first
- Write your answer on the piece of paper

Spoiler Alert

- The next slide gives away the answer to last week's MU puzzle
- Want to try it yourself first? Stop here
 - Starter code: `mu_puzzle.dfy` in the extra folder on the course GitHub
 - Full solution: `mu_puzzle_answer.dfy` in the same folder

Extra - MU Puzzle Answer

- Last week: start with MI. Can you ever reach MU?
- Answer: no
- Invariant: the number of Is is never a multiple of 3
 - MI has 1
 - Rule 2 doubles the count, Rule 3 subtracts 3, Rules 1 and 4 keep it
 - None of these turns a non-multiple of 3 into a multiple of 3
- MU has 0 Is, and 0 is a multiple of 3
- “Not MU” alone is not inductive: MIII $\rightarrow$ MU